Continual Learning & Memory Models

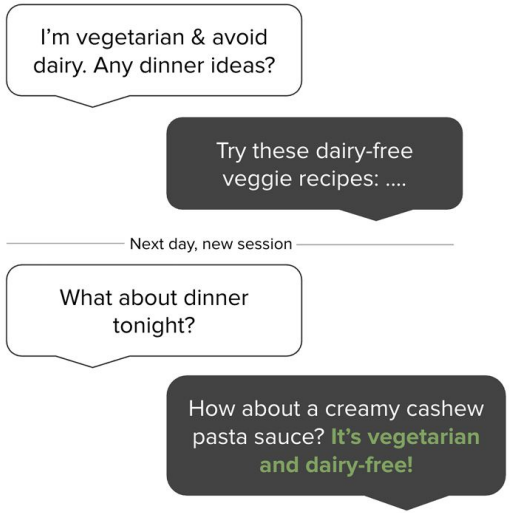
COMS 6998

Week #9: March 25, 2026

Paper title: Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory

Motivation of Scalable Long-Term Memory

- AI agents cannot persist information across separate sessions nor after context overflow
- Agents tend to forget users preferences, repeat questions, contradict previously established facts
- Context length merely pushes these problems further down but does not remove them
- Goal: Build a scalable memory system that stores only salient information



Author: Gilbert Yang

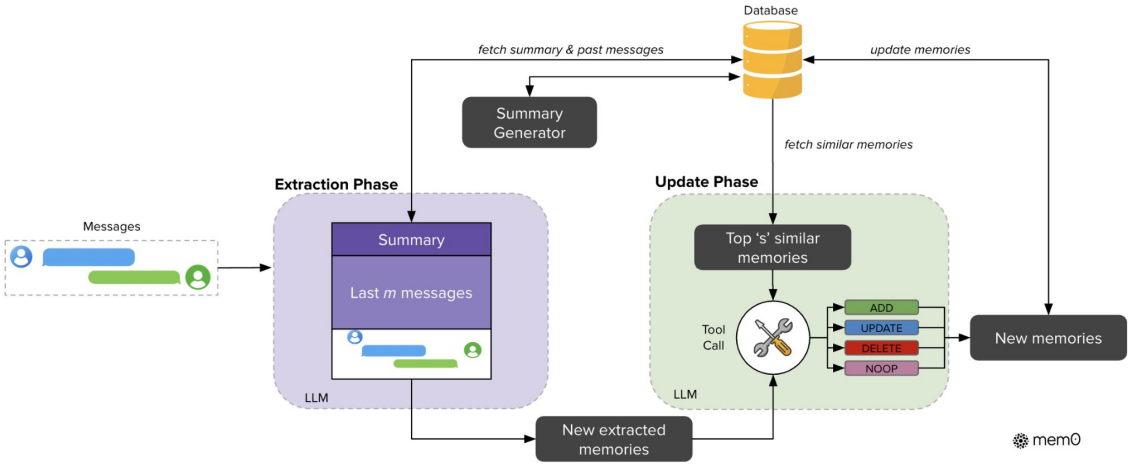
Mem0

Step 1: Extraction

- Input: new message pairs, summary of prior conversation, last m recent messages
- LLM extracts salient candidate memories

Step 2: Update

- Retrieve top-s similar stored memories
- Use LLM to choose 1 action: ADD, UPDATE, DELETE, NOOP



Author: Gilbert Yang

Mem0^{g}

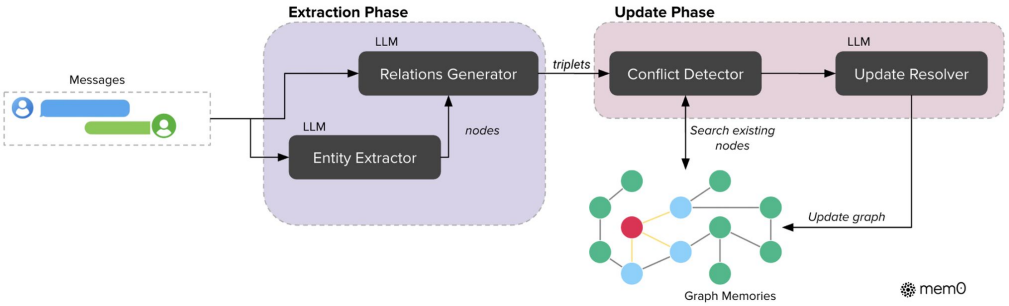
Why graphs?

- User facts are relational, not isolated

Pipeline:

1. Entity extractor identifies entities + types
2. Relation generator converts conversation into triplets
3. Store memory as directed labelled graph
4. Match entities to existing nodes by semantic similarity

Note: we mark obsolete relations as invalid rather than deleting



Author: Gilbert Yang

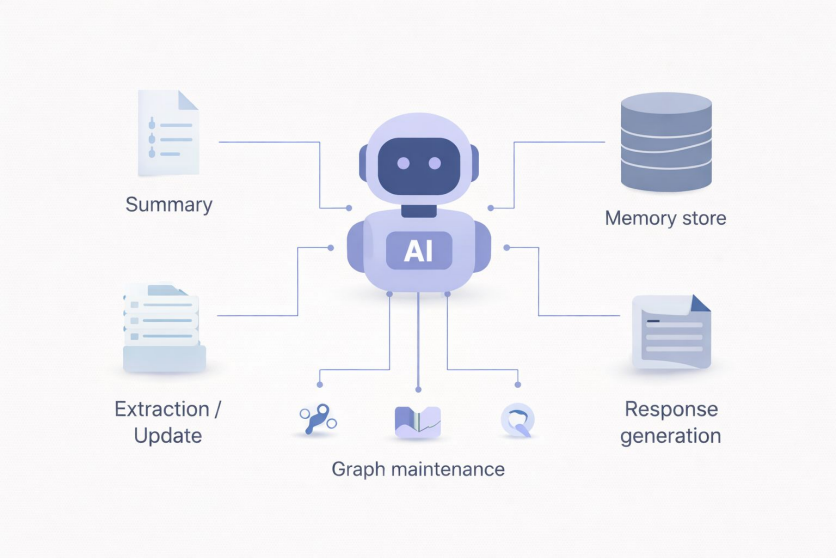
An LLM-OS for Memory

***Mem0 is an LLM-first* memory orchestrator**

- Base LLM does memory CRUD ops
- Generates content summary
- Performs graph maintenance
- Final response generation

A novel system design with many benefits

- Modular design with minimal retraining overhead
- Plug and play deployment for different foundation models
- Direct beneficiary of stronger base LLMs



Author: Peter Driscoll

Mem0 system design advantages

Mem0 wins on single-hop and multi-hop questions

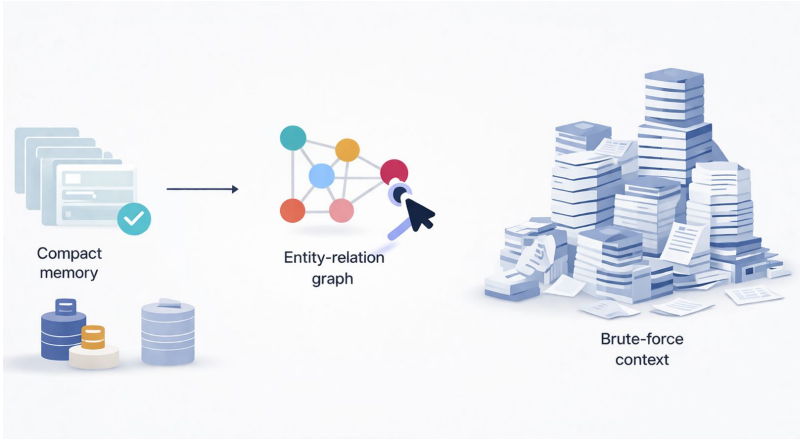
- Semantic vector-store memory is enough when combined with LLM operator
- Dense embeddings power low latency retrieval

Mem0g is strongest on temporal & relational tasks

- Injects structure via explicit entity-relation graph, most useful when relations and event order matter
- Overall J gain (+1.56) driven by temporal (+2.62) and open-domain (+2.78)

Memory conservation

- Full-context: 26,031 tokens
- Mem0g: 3,616 tokens
- Mem0: 1,764 tokens



Author: Peter Driscoll

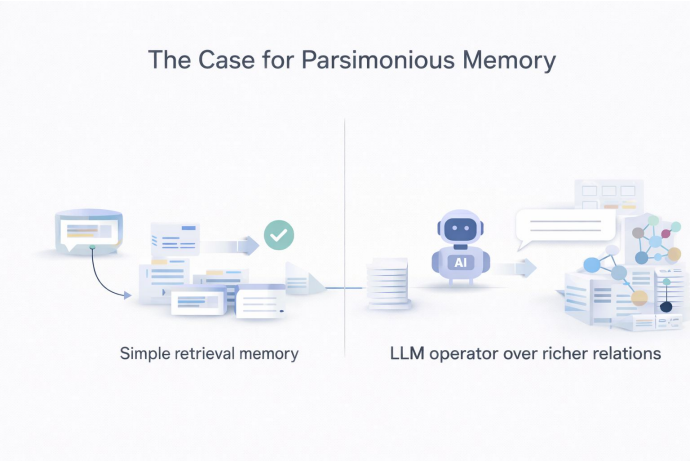
The Case for Parsimonious Memory

Selective memory with LLM operator beats brute-force

- Longer context windows can help, but they do not create persistent long-term memory on their own
- Compact external memory keeps retrieval efficient, while the LLM provides the semantic flexibility to extract, update, and synthesize across sessions
- Mem0 shows this synergy in the simplest form; Mem0g adds structure only when temporal or relational reasoning demands it

System lesson

- Keep persistence in external memory store
- Let LLM handle semantic control and response generation



Author: Peter Driscoll

Summary / Core Contributions

- Two memory centric architectures with clear pipeline from raw text to memory management automated via LLM
- Dynamically extracts salient facts from conversations via LLM prompting
- Performs memory management via LLM-based operations (ADD / UPDATE / DELETE / NOOP)
- Production Efficiency: Achieving a 91% lower p95 latency and over 90% savings in token costs

Reviewer: Saad Satter

Strengths

- Evaluates across multiple baselines (RAG, MemGPT, OpenAI memory, etc.)
- Covers diverse tasks: single-hop, multi-hop, temporal, open-domain
- Portable architecture: Plug and play components (LLM, Vector DB, Database, etc.)
- Mem0 and Mem0g outperforms most tasks with high efficiency
- Honest and clear tradeoffs between certain shortcomings

Reviewer: Saad Satter

Weaknesses

- Mem0g No strong advantage: Base Mem0 did better in most tasks and nearly as well in temporal

- Would be nice to have some ablation study:
  - Effect of summary vs no summary
  - Effect of m (context window) and s (retrieval size)
  - Contribution of each module

Weaknesses - Cont.

- LOCOMO is a 10 conversation dataset which is fairly small
- Experiments on failure cases:
  - What guarantee that the LLM doesn't hallucinate in the update phase?
- Were the baselines used the best baselines? (use a more sophisticated RAG)

Score

- **Quality: 3/5 -** Solid pipeline for memory management but lacks rigorous testing
- **Significance: 3/5 -** Has clear industry application
- **Clarity: 4/5 -** Well written, a little repetitive
- **Novelty: 3/5 -** Combines standard components (RAG, summary, LLM CRUD) and using the Graph architecture provided no meaningful breakthrough
- **Overall: 3.5/5 -** Accept

Reviewer: Saad Satter

The Three-Component Framework

Every memory system in the literature decomposes into exactly these three layers

01

Memory Storage

How raw information is kept

- Raw interaction traces
- Summaries & structured notes
- Knowledge graphs
- Typed storage tiers

02

Memory Management

How stored info is maintained

- Forgetting curves
- LLM-driven ADD / UPDATE / DELETE
- Linkage & consolidation
- NOOP option

03

Memory Integration

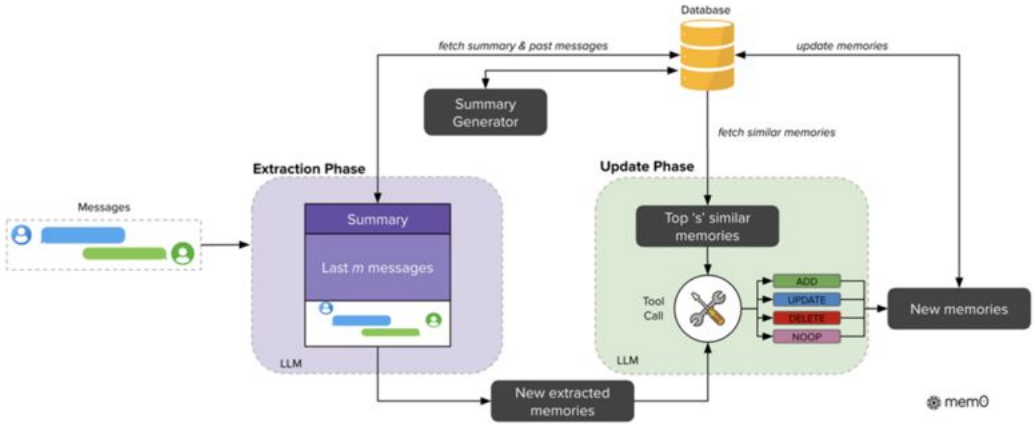
How info re-enters the LLM

- Full context stuffing
- Semantic top-k retrieval
- Temporal filtering
- Agentic iterative retrieval

Archeologist: Suhas Morisetty

Mem0 Architecture

Extraction Phase → Update Phase → Retrieval



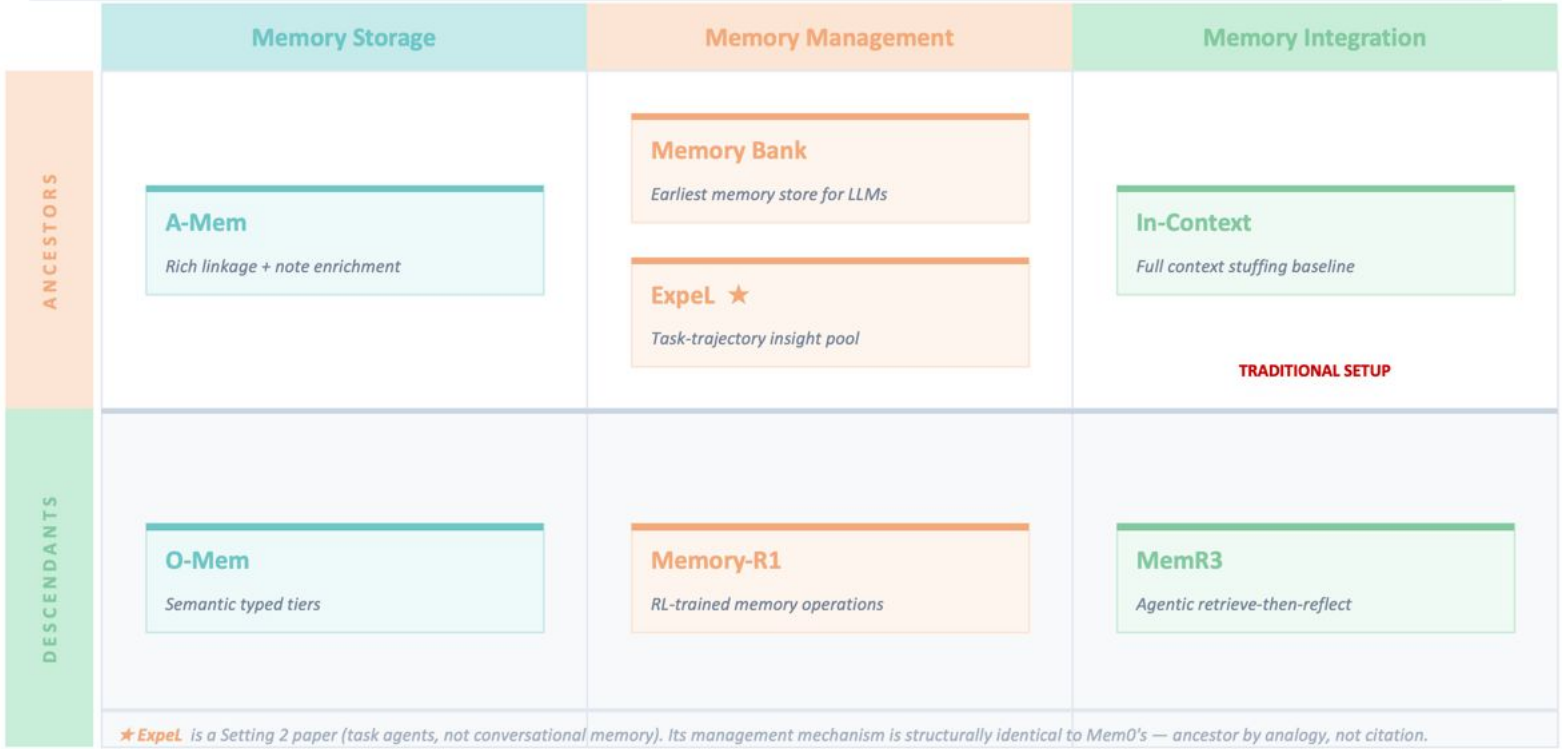
Retriever Integration
→ Top-k retrieval

Natural Language Strings
Simple summaries

Archeologist: Suhas Morisetty

Memory Systems Landscape

Papers mapped across the three-component framework · ancestors above, descendants below



Archeologist: Suhas Morisetty

Two Settings Where Memory Benchmarked

SETTING 1

Factual Personal Memory

Conversational Agents

Goal

Remember facts, preferences & events across sessions

Benchmark

LoCoMo — long multi-session dialogues

Q types

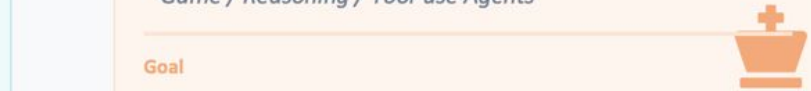
Single-hop · Multi-hop · Temporal · Open-domain

← Mem0 lives here

SETTING 2

Self-Evolving Task Agents

Game / Reasoning / Tool-use Agents



Goal

Get better at recurring tasks by learning from trajectories

Memory

Generalizable strategies & insights from past attempts

Examples

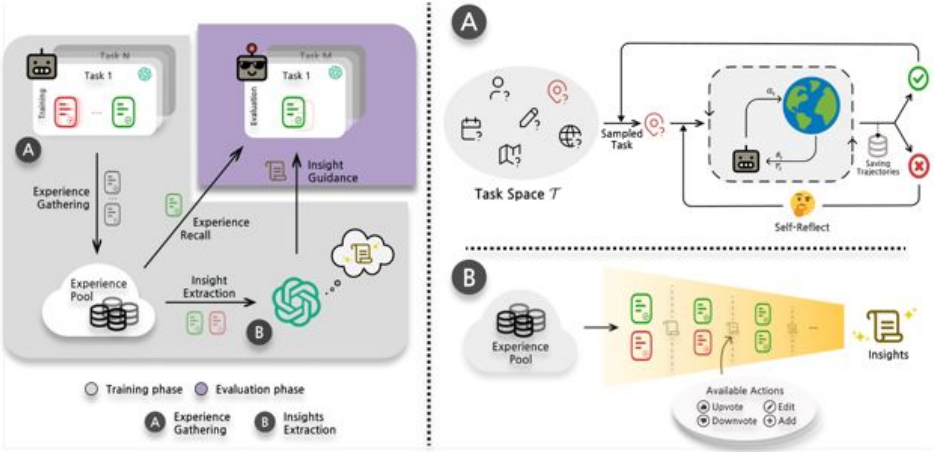
Math reasoning · Game playing · Tool-use agents

← ExpeL ancestor lives here

Archeologist: Suhas Morisetty

ANCESTOR *Management Layer*

ExpeL: LLM Agents Are Experiential Learners

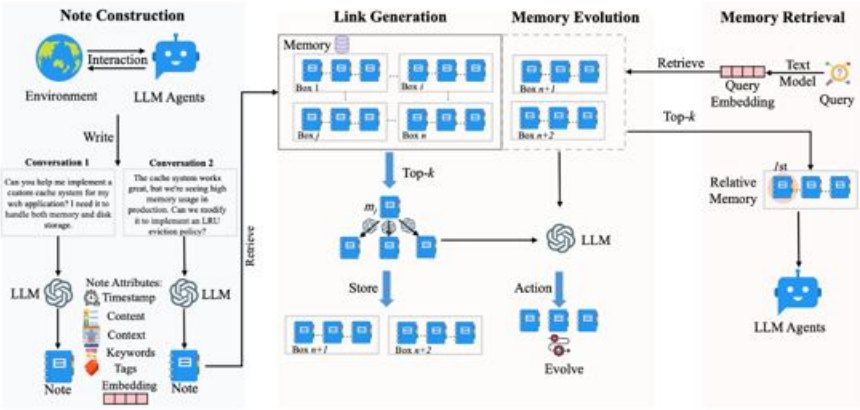


LLM as consolidation arbiter clearly inspired Mem0

Archeologist: Suhas Morisetty

ANCESTOR *Storage Layer*

A-Mem vs Mem0 — The Complexity Trap



Keywords + Tags

LLM enriches every note with metadata

Link Generation

Top-k similar notes get linked on update

Memory Evolution

Historical notes rewritten on new memory

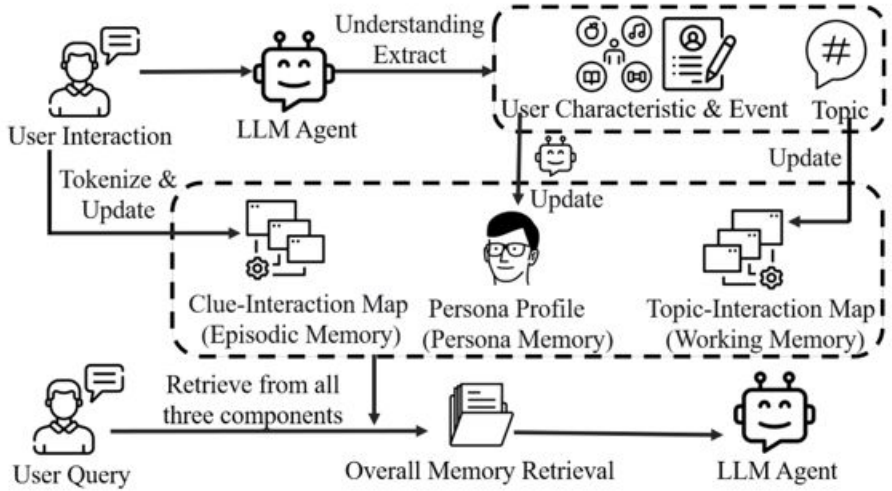
⚠ Problem

Compounding cost & update noise corrupts memories

Archeologist: Suhas Morisetty

O-Mem — Semantically Typed Memory Tiers

Persona Memory



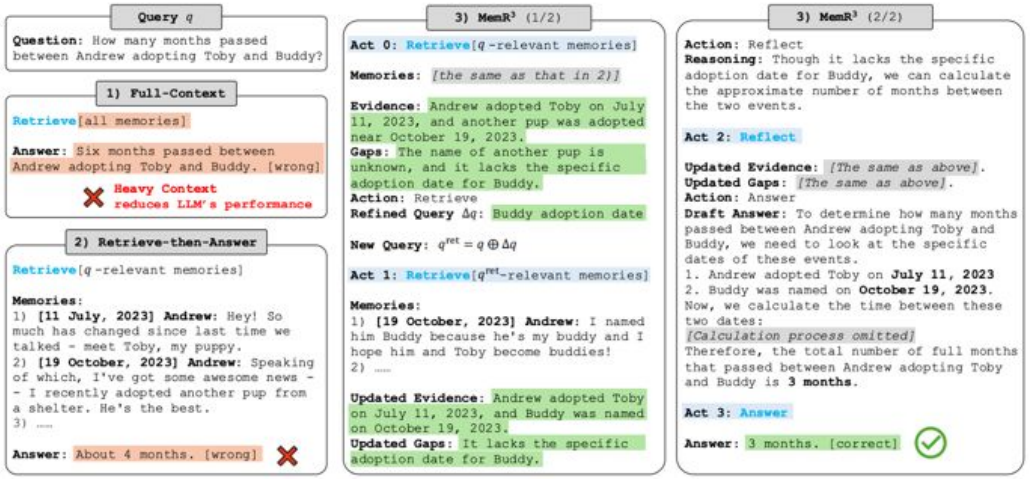
Persona Memory
Long-term stable user attributes, factual events & preferences

Working Memory
Topic-indexed recent interactions & active context

Episodic Memory
Associative recall triggered by keywords & context cues

Archeologist: Suhas Morisetty

MemR3: Memory Retrieval via Reflective Reasoning



LoCoMo · LLM-as-Judge (GPT-4o-mini)

75.54% → 81.55%

RAG baseline → MemR3 · surpasses even Full-Context

Evidence / Gap Tracker

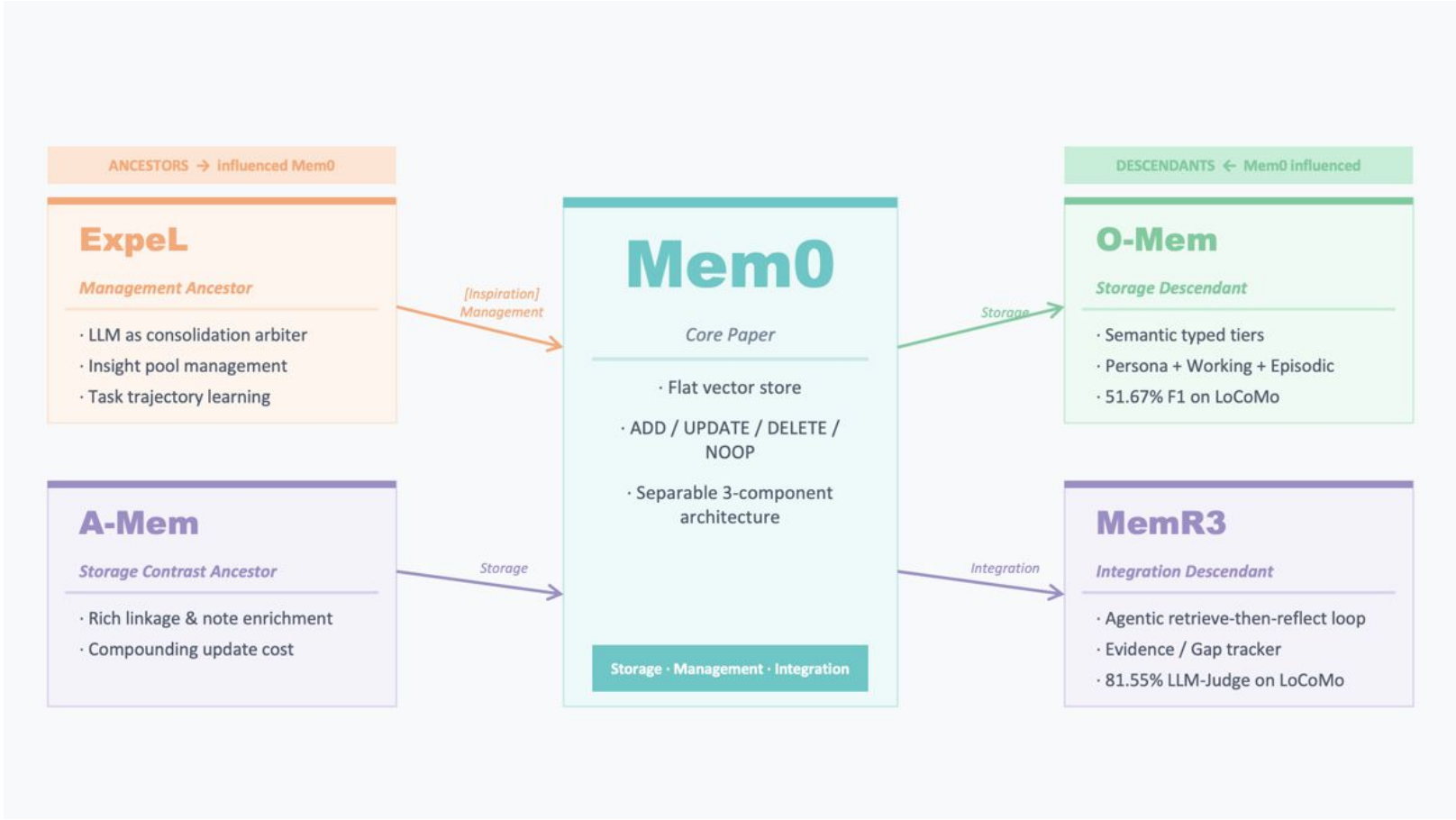
What do we know? → Evidence set
What are we missing? → Gap set
Terminates when gap = ∅

Backend-agnostic

Wraps Mem0, RAG, or Zep equally — validates that integration is truly separable from storage & management.

Key Insight:

Mem0 solved storage & management. MemR3 treats integration itself as an agentic reasoning problem with explicit state — the missing third piece.



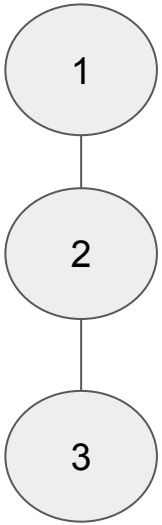
Archeologist: Suhas Morisetty

Before Mem0: the field knew LLMs needed memory

Longer context helped - but it did not create memory

By 2025, the field had already moved from bigger prompts toward persistent persistent memory for multi-session dialogue

- facts get buried in long chat histories
- sessions drift and preferences are forgotten
- full context is expensive and noisy



Bigger windows
more tokens, but still no persistence

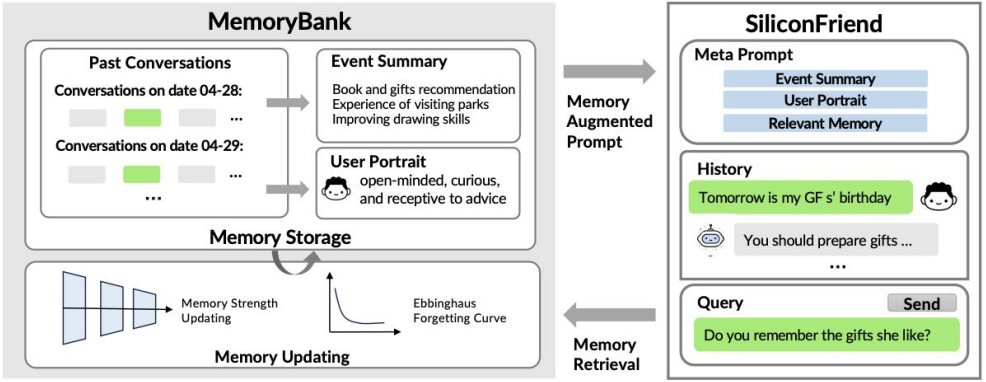
RAG over chat logs
retrieve old chunks when needed

External memory systems
store conversation outside the prompt

Archeologist: Tanya Mehta

Ancestor → MemoryBank Enhancing Large Language Models with Long-Term Memory

- **Goal**
  - give LLMs long-term memory for sustained interaction
  - carry user history and traits beyond prompt
- **Approach**
  - store dialogue logs, event summaries, and a user portrait
  - retrieve relevant memory with dense vector search + FAISS
  - update memory with a forgetting-curve inspired rule
- **Result:** recalled past details over 10-day conversation and 194 probes



Key contribution: memory became separate system around LLM

Archeologist: Tanya Mehta

Why the field was ready for Mem0

MemoryBank proved persistent memory helps

But memory still looked more like stored conversation than managed facts

What earlier systems established

- kept memory outside the prompt
- retrieve old context when useful
- adapt to a user over many sessions

next step

What still felt missing

- salient fact extraction
- clean handling of contradictions
- explicit memory editing over time

Mem0's shift

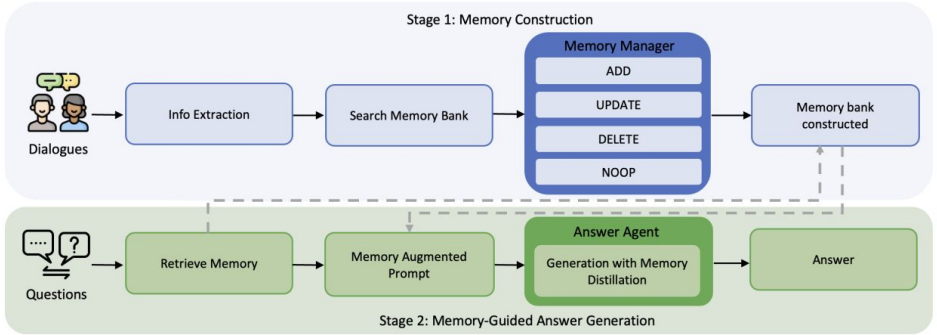
ADD | UPDATE | DELETE | NOOP

extract salient memories -> consolidate them -> retrieve compact memory instead of raw history

Archeologist: Tanya Mehta

Descendant → Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning

- **Goal**
  - make external memory learned, not heuristic
  - fix two decisions: what to update and what to use
- **Approach**
  - RL-tune 2 agents: Memory Manager + Answer Agent
  - Manager pics operation
  - Answer agent distills retrieved memories before answering
  - reward = downstream QA correctness
- **Result:** Outperforms Mem0 and other earlier memory systems



manager policy

$$(o, m') \sim \pi\theta(\cdot \mid x, \mathcal{M}_{\text{old}}),$$

answer policy

$$y \sim \pi_\theta(\cdot \mid q, \mathcal{M}_{\text{ret}}).$$

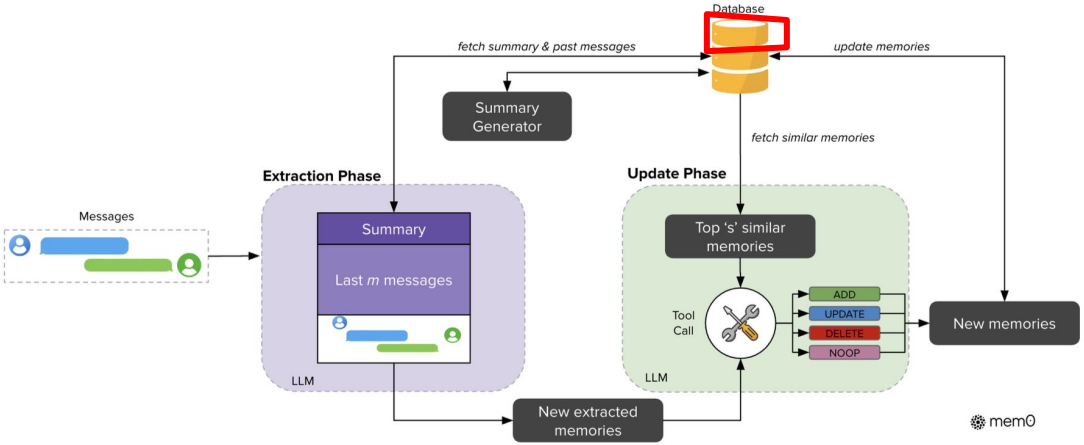
Archeologist: Tanya Mehta

Predictive Memory Maintenance: Learning Staleness Dynamics in Long-Term Agent Memory

Researcher: Justin Hou

The Gap in Mem0's Update Model

- Mem0 updates memory reactively — ADD, UPDATE, DELETE, NOOP
- A memory is trusted until contradicted by new evidence
- But real-world facts go stale at different rates with no contradiction ever arriving



Researcher: Justin Hou

The Proposal: Learned Staleness Dynamics

***Core idea**:* model confidence of each memory as a function that decays over time — and learn the decay rate from data rather than hand-coding it.

Formally: attach a confidence function:

- $c(m, t) = Decay(m, t) \cdot Neighborhood(m, t)$
- $Decay(m, t) = c_0 \cdot \exp(-\lambda(type, domain) \cdot \Delta t)$
- $Neighborhood(m, t) = 1 - \alpha \cdot S(m, t)$

to each memory edge in Mem0g's graph

Key properties:

- Fact type determines base decay rate — learned per domain
- Time since initialization drives decay
- Weak contradicting signals (related facts changing nearby in the graph) accelerate decay even without direct contradiction

Researcher: Justin Hou

Closing the Loop: Proactive Verification

Staleness detection alone isn't enough — you need an **update mechanism**.

Proposed**: verification scheduler driven by expected **information gain

When `c(m, t)` drops below threshold:

1. Scheduler identifies lowest-confidence memories
2. Ranks by expected information gain — memories whose staleness would most affect downstream agent behavior get prioritized
3. Agent issues lightweight verification queries against available evidence streams
4. Outcome feeds back into Mem0's existing ADD/UPDATE/DELETE pipeline

*This reframes memory management from **reactive update** to **predictive maintenance***

Researcher: Justin Hou

Research Contributions and Open Problems

***Contributions**:*

- Learned staleness model over Mem0g's graph edges — replaces binary validity with confidence dynamics
- Verification scheduler using expected information gain
- New evaluation metric: staleness recall — does the system flag memories that have become wrong before they cause downstream failures?

***Hard open problems**:*

- What's the right training signal for staleness rates?
- How do you handle correlated staleness: When one memory going stale implies others likely have too?
- Benchmark construction: LOCOMO has no time-varying ground truth

Researcher: Justin Hou

Static Salience for Dynamic Memory: Mem0 dynamically learns, but the way it learns remains static

- Mem0's Extraction Phase relies on a **fixed prompt** that assumes salience is task- and context-agnostic
- Mem0's update decisions (ADD/UPDATE/DELETE/NOOP) are made via **static single-shot LLM judgments** with no feedback or dynamic learning
- The memory store evolves, but the **policy governing memory formation remains fixed**, creating an architectural **opportunity for an adaptive ("meta") memory feedback loop**

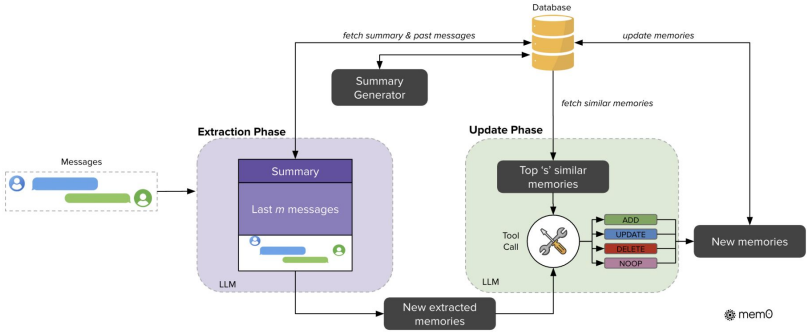
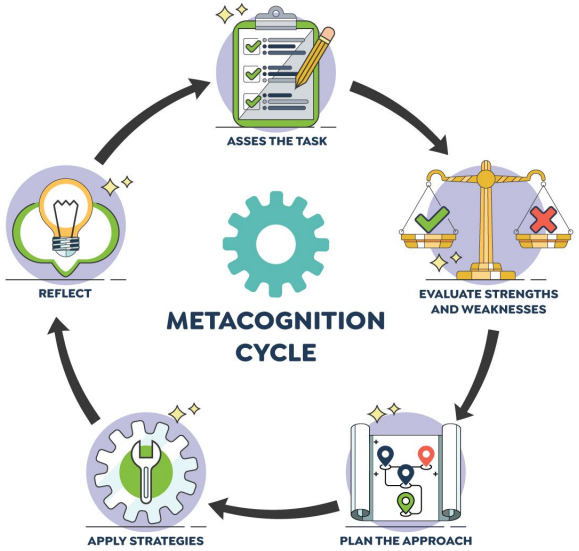


Figure 2: Architectural overview of the Mem0 system showing extraction and update phase. The extraction phase processes messages and historical context to create new memories. The update phase evaluates these extracted memories against similar existing ones, applying appropriate operations through a Tool Call mechanism. The database serves as the central repository, providing context for processing and storing updated memories.

Researcher: Tevin Kim (tk2999)

Extending Mem0's Core Assumption: If memory can be governed by LLMs, it becomes a prompt optimization problem

- Mem0 establishes the idea that memory extraction and updates can be reasoned by prompted LLMs
- This implies that, even if Mem0's system prompts are static, **memory behavior is controllable through prompting** rather than fixed architecture
- Thus, **memory formation and intake can be framed as a prompt optimization problem via meta prompting**



[1]Image Credit

Researcher: Tevin Kim (tk2999)

Self-Adapting Meta-Memory Prompts (“MMPs”): A periodic reflection step learns from usage to adapt guidelines for extraction and updates

- Keep Mem0’s original extraction and update system prompts, but **append dynamic guideline sections** (MMP-E for editing, MMP-U for updating) both initialized with generic rules
- Every k message pairs, a separate LLM reviews the last k message pairs to **identify retrieved memories and patterns** in what types of information are actually useful to inform MMP updates
- The scope of **MMP-E is limited to the definition of salience** and **MMP-U to the aggressiveness of ADD/UPDATE/DELETE decisions**

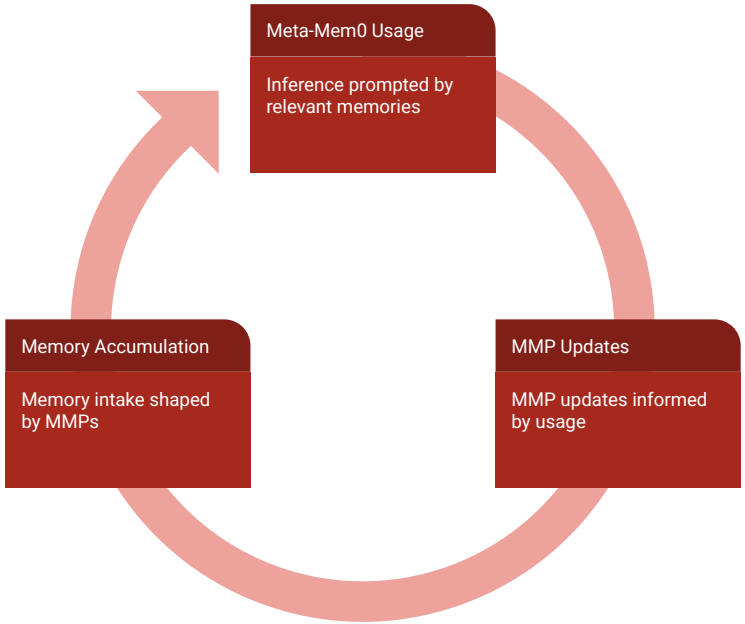
Hypothetical MMP-E

```
Prioritize storing:
- User’s standing preferences and routines (e.g., vegetarian diet,
gym schedule, work hours)
- People in the user’s life and their relationships (e.g., “Alex =
coworker”, “Maya = roommate”)
- Plans that are referenced across sessions (e.g., “trip to Boston
next weekend”, “weekly team meeting”)

Deprioritize storing:
- One-time activities with no follow-up (e.g., “had lunch at
Sweetgreen on Tuesday”)
- Exact timestamps unless tied to recurring or future plans
- Generic conversational filler (e.g., opinions, small talk
without future relevance)

Context-specific adjustments:
- If a person is mentioned more than once → start treating them as
persistent (store relationship)
- If an activity repeats → upgrade it to a routine (store with
higher priority)
- If a plan is referenced again → store full details (who, when,
where)

Recent learned patterns:
- Dietary preferences are repeatedly needed for recommendations →
always store immediately
- Friends/coworkers frequently reappear in questions → prioritize
storing names + relationships
- One-off meals and isolated dates are never reused → avoid
storing unless repeated
```


Researcher: Tevin Kim (tk2999)

Adaptive Memory Policy as a Feedback Loop: The evaluation and update phases become usage-driven, self-improving flywheels

- MMPs turn the Mem0 architecture into a **feedback loop** where future outputs generate implicit signals that refine memory intake
- This converts memory formation from one-shot reasoning to **adaptive prompting** where the system learns which types of information have persistent downstream value
- As a result, memory behavior becomes **trajectory-dependent and self-specializing**



Researcher: Tevin Kim (tk2999)

Configurable, Self-Specializing Memory with MMPs: MMPs allow controllable trade-offs between memory efficiency, performance, and specialization

- Depending on meta-prompt configuration, MMPs can **reduce storage needs** with more selective memory accumulation, **improve performance** by prioritizing higher downstream-utility, or optimize across both
- Memory policies organically become **self-specializing** as the feedback loop implicitly rewards domain- and user-specific memory intake based on observed usage patterns
- If efficacious, meta-prompting memory intake can become a design space where developers can **steer agent "cognitive" patterns** within a modular architecture

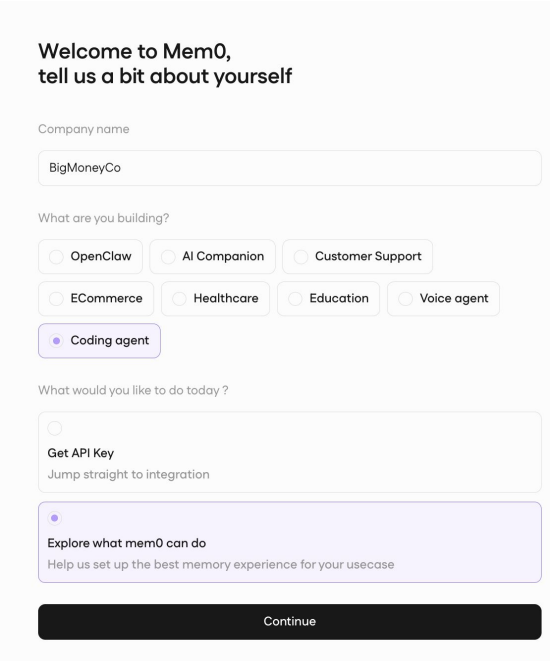
Researcher: Tevin Kim (tk2999)

Reproducing mem0

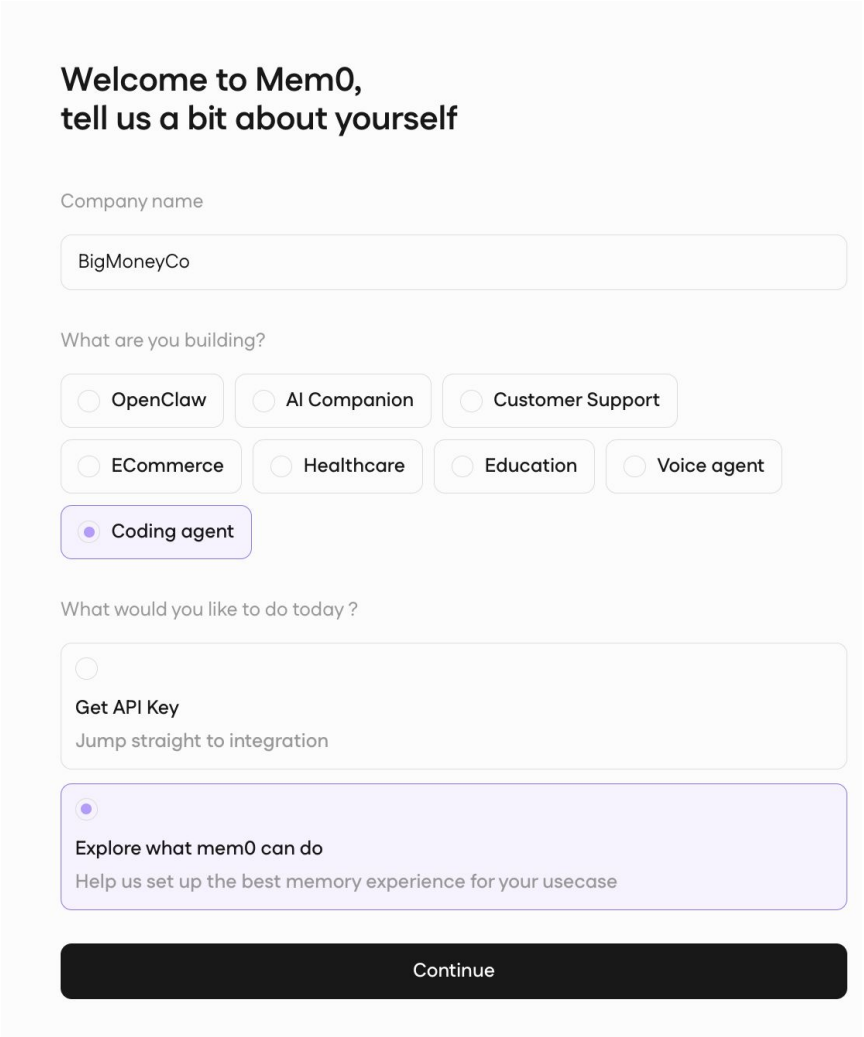
Hacker: Maksym Bondarenko

Goal: faithful reproduction of paper results

Secondary goal: how easy is their framework to reuse?



<WC>: <presenter name>



<WC>: <presenter name>

Remember Me: A Persistent AI Companion for Elder Care

Built on Mem0 Adaptive Memory

The pitch:
We are a startup building an AI elder care companion that uses Mem0's persistent, updatable memory to know patients across every conversation — their medications, family members, preferences, and history.

The problem:
54M Americans live with dementia. Current AI resets every session. No product today maintains true long-term patient memory.

The ask:
Fund a 6-month pilot with 3 care facilities to validate memory accuracy, safety, and caregiver satisfaction.

Industry Practitioner: Pranati Modumudi

The Problem

Why existing AI companions fail elder care patients

AI has no memory across sessions.

Every LLM-based companion resets after each conversation. It can't recall a patient's name, medications, or what they talked about yesterday. For dementia patients, this feels like talking to a stranger every day.

Care is fragmented.

Doctors, aides, and family each hold a piece of the patient's picture. No single system synthesizes that history. Shift changes alone cause critical context loss.

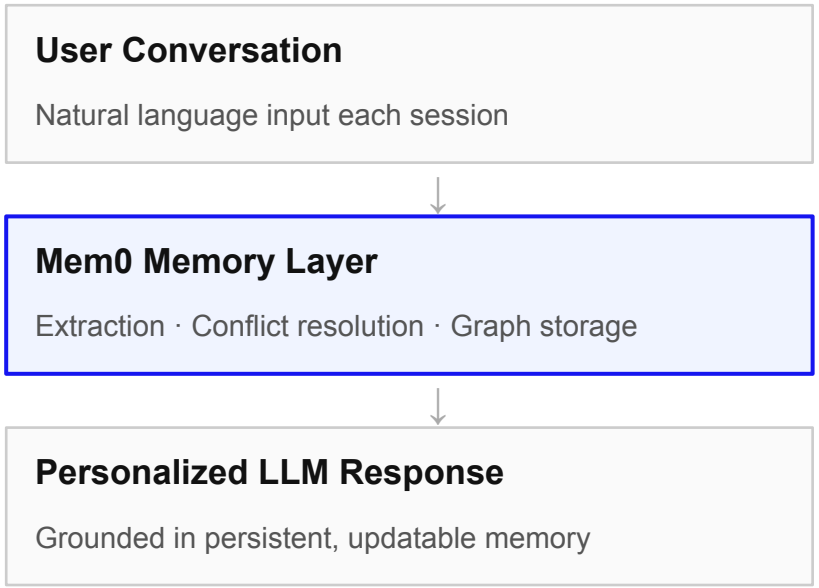
The scale is enormous.

54M Americans live with Alzheimer's or related dementias — projected to double by 2050. 67% repeat the same stories multiple times daily because their care loop lacks shared memory.

Industry Practitioner: Pranati Modumudi

Why Mem0 Makes This Possible

The key technical contribution of the paper



Persistent Memory

Facts, preferences, medication history survive across sessions indefinitely

Auto-Updating

Contradictions resolved; stale info replaced — no manual curation

Multi-Level Scoping

Patient, caregiver, and family each have distinct memory views

Industry Practitioner: Pranati Modumudi

Honest Tradeoffs

One positive impact, one negative impact, and key concerns

✓ Positive Impact

- Reduces caregiver burden — less repetition, better context across shifts
- Continuity of care: every provider sees the same patient history
- Early detection: memory drift logged over time → potential dementia signal
- Dignified experience: the AI knows who you are

✗ Risks & Concerns

- False memories — Mem0 can mis-extract facts; dangerous for vulnerable users
- Privacy — ultra-sensitive health + behavioral data; breach risk is severe
- Over-reliance — families may substitute AI for human contact
- Consent — patients with dementia cannot meaningfully opt in

Industry Practitioner: Pranati Modumudi

1. Could this architecture be adapted to multimodal sensing (ex: video, audio) instead of just text?
2. What are some metrics used in deployment to see if this is ready to use in practice?

Questions/Comments: Emily Bejerano

The paper's core mechanisms (retrieval, consolidation, and ADD/UPDATE/DELETE decisions) all hinge on semantic similarity, but the distance metric and similarity threshold are not defined nor ablated. Meanwhile, what counts as a salient memory in Mem0 or an important entity in Mem0g is delegated entirely to LLMs, via prompts that are never shown. Without formal criteria for memory quality, how do we know that the system will remember the right things if tested on conversations that don’t follow the LOCOMO question format?

Questions/Comments: Vici Milenia

1. While the paper shows impressive results in the provided evaluation environment, I wonder if its methods will effectively scale when task context reaches over one million tokens? The authors emphasize that the real benefits of the approach will be apparent compared to full-context prompting when the context is very long, but because the method still relies on embedding-based retrieval, I wonder whether its performance will start to degrade?
2. The memory storing process seems to be highly reliant on LLMs synthesizing the correct facts from given queries. I wonder if this increases the risk of hallucinated memories being stored over time?

Questions/Comments: Daniel Ben-Levi

1. Why do you think Zep beats Mem0 in the open domain tests? What is it about adding external knowledge that makes the performance more generalizable? Is there a mechanistic explanation as to why Mem0 lags here?

2. You state Mem0 saves more than 90% token cost, how much appresivity are you giving up for that efficiency? For agent-based tasks that need to be precise (due to combinatorial explosion of errors being magnified) are you worsening your utility for agent-based tasks where high precision is required?

Questions/Comments: Christian Montgomery

1. To the authors, you frame Mem0 as achieving near-competitive quality at a fraction of the cost, but there's a persistent ~4-6 point J gap to full-context. As context windows continue to grow (Gemini at 10M tokens, future models likely larger), and as inference costs drop (w/ quantization, speculative decoding, *cheaper hardware…?*), the economic argument for memory extraction may erode faster than the quality gap closes. But the paper doesn't really model this trajectory. Under what assumptions about context window scaling and cost reduction does an external memory system remain the better engineering choice?

2. As someone who has tried integrating Mem0 in applications (also corroborated by some folks on Reddit), I found that adding memories worked fine, but retrieving them was unreliable: queries often returned nothing or missed obvious matches. One Redditor running voice-based AI agents for small businesses (mining, construction, shipping) found Mem0 useful for "transient knowledge" but noted the system required careful orchestration. Another classmate (Christian) above asked about Zep and how it's much better for basic retrieval reliability. Mem0 evaluates on LOCOMO where conversations are clean, structured, and relatively short (~26K tokens). Real-world data is noisy, domain-specific, and orders of magnitude larger. The paper's vector similarity threshold for retrieval and the entity-matching threshold in Mem0^g are likely tuned for the benchmark distribution. How sensitive are these thresholds, and did the authors do any robustness analysis?

Questions/Comments: Shayan Chowdhury

1. How could Mem0 be extended to multi-agent systems?
    a. What would be the tradeoffs in maintaining separate memory stores per agent versus a shared memory across different agents? Or if a hybrid approach was taken, what could be the criteria to decide which memory to use?
    b. For a shared memory across agents, what strategies could be used to handle conflicts in the memories from different agents?

Questions/Comments: Kirthana Natarajan

1. Mem0 treats memory as a flat set of facts, while Mem0^{g} adds relational graph structure but the graph memory actually hurts performance on multi-hop questions compared to base Mem0. Why is this the case and what does this tell us?
2. Mem0 uses an LLM to classify each extracted fact as ADD, UPDATE, DELETE, or NOOP rather than a dedicated classifier trained for this task. What are the tradeoffs of this design choice and does it make sense?

Questions/Comments: Zach Zusin